How simblocks works

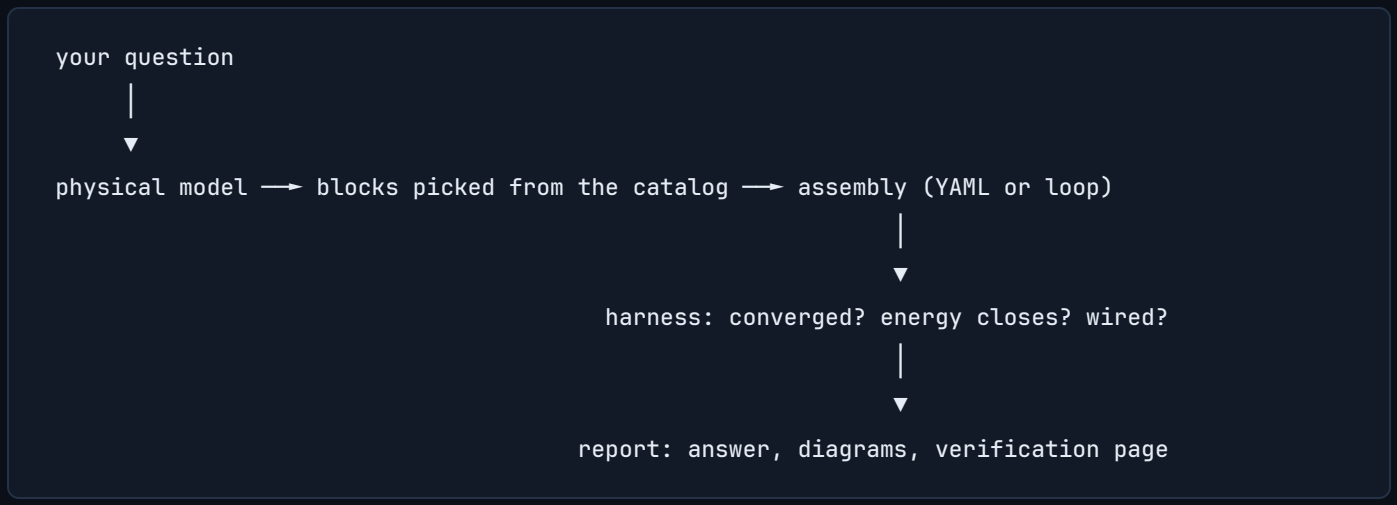
This page explains what happens between "simulate this hit" and the report: how the pieces are built, how they connect, how data moves every time step, and how the result is checked. It is for the curious. You do not need any of it to use the skill.

---

1. The big picture

A simulation here is **assembled, not written**. Three layers do the work:

LAYER	WHAT IT IS	WHERE IT LIVES
Blocks	Small tested functions. Each one does one piece of physics, with a citation.	`scripts/simblocks/`
Assembly	A YAML system file (compiled automatically), or a hand-written loop for 3D or special cases.	`*.yaml`, case-study `.py` files
Harness	A battery of checks that every assembled sim must pass before its numbers are trusted.	`simblocks/diagnostics/verify.py`



The point of the split: **physics is written once and tested once**, then reused. A new sim is mostly wiring, and wiring is the part the harness checks.

---

2. Blocks

Every block is a Python function registered in a catalog with a fixed address:

```
domain / category / geometry / load        e.g.  structures/beams/cantilever/end_point
```

and a **source** (Roark table and case, a textbook equation, a paper). There are about 310 of them across 12 domains: materials, structures, contact, constitutive, friction, actuators, rigid body, geometry, aero, thermal, integrators and diagnostics.

Blocks come in **three kinds**:

KIND	WHEN IT RUNS	WHAT IT DOES	EXAMPLE
Builder	Once, before the sim starts	Turns geometry and material into numbers	`beam_plastic_spring`: 30 × 6.35 mm AR500 bar becomes a stiffness and a yield table
Kernel	Every time step, inside the loop	Turns the current state into a force (or current, or voltage)	`hertz_sphere_force`, `coulomb_tanh`, `esc_supply`, `ecm_constant_power`
Analytic	On demand	A closed-form answer, used for sizing and for checking sims	`spin_up_time`, Roark deflection formulas

Kernels are deliberately simple. They take plain numbers and return plain numbers, so they can be compiled to machine code (numba) and run millions of times per second. A kernel never knows what it is connected to. A contact kernel only sees "how far am I overlapping, and how fast" and answers "this much force".

Every block is tested against an independent answer before it goes in the catalog: a hand calculation, a published table, or a second method. The tests live in `scripts/tests/` (about 195 of them).

---

3. Materials

Materials feed the builders. `load_curve("7075 whatever temper")` resolves loose text to a catalog entry and returns a **published stress-strain fit** with its citation. A curve file you drop in `materials/curves/user/` overrides it.

Composite plates go through classical laminate theory: `laminate_card("T300/5208 [0/+-45/90]2s")` returns the full stiffness matrices, first-ply-failure strengths and warnings about coupling.

Builders then turn the curve into what a sim needs. For example, a bending arm becomes a **moment-curvature table**, which becomes a force-deflection table with a yield point and hardening. The sim never sees the curve itself, only the table the builder made from it.

---

4. Assembly with a system file

For models where each body has **one coordinate** (a translation or a rotation), you describe the model in a YAML file and the compiler builds the sim. No loop is written by hand.

```
bodies:
  rotor: {dof: rotation, inertia: 8.5e-4 kg m^2, v0: 12000 rpm}
  hub:   {dof: translation, mass: 0.40 kg}
  wedge: {dof: translation, mass: 0.15 kg}

elements:
  tooth: {type: hertz, measure: hub + 0.095 * rotor - wedge, K: 2e10, n: 1.5}
```

4.1 The measure: how links are made

Every element acts along a **measure**: a straight sum of body coordinates.

```
s = c1 * q1 + c2 * q2 + ... + constant
```

`s` is how far the element is stretched or compressed. The coefficients are the **lever arms**. A plain spring between two bodies has coefficients +1 and -1. A blade tip has the blade radius in front of the rotation angle.

The compiler turns every measure into **one row of a table,** `C` : one row per element, one column per body.

ELEMENT	ROTOR	HUB	WEDGE
tooth	0.095	1	-1

That row is the whole connection. It is built once, before the run, and never changes.

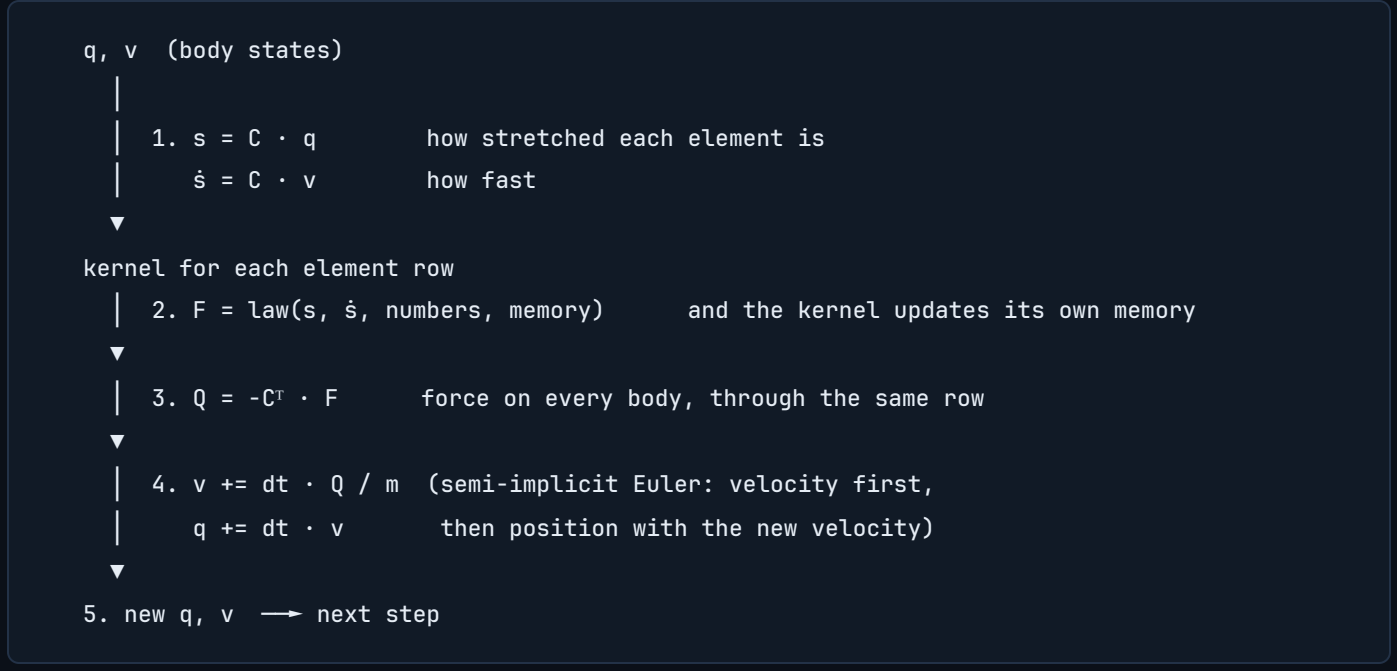
4.2 What the compiler builds

ARRAY	HOLDS	CHANGES DURING THE RUN?
`q` , `v`	Position and velocity of every body	Yes, every step
`m`	Mass or inertia of every body	No
`C`	The link table	No
`typ`	Which kernel each element uses	No
`P`	Each element's numbers (stiffness, yield table, motor Kv...), filled by builders	No
`st` , `aux`	Each element's memory (plastic set, battery charge, ...)	Yes, by its own kernel

---

5. One time step

Every step (from tens of nanoseconds for impacts to a tenth of a millisecond for a battery fight) does the same five things:



Nothing tells a spring how compressed to be. Compression comes from where the bodies are. The force comes from the compression. The motion comes from the force. Peak forces and deflections are outputs, never inputs.

Why the same table is used in both directions: the force an element puts on body A and the reaction it puts on body B come from the same row of `C`. They cannot disagree. A lever arm used to compute the stretch is automatically the lever arm used to apply the torque. This removes the most common hand-written bug, a reaction applied at the wrong place or not at all.

Kernels never call each other. Each reads its own row's `s` and `ṡ`, its own numbers and memory, and returns a force. That is why a block can be dropped into any model.

Example: a blade tooth hitting a wedge

1. The tooth has not reached the face: `s` is negative, the contact kernel returns 0, nothing happens.
2. Next step the tooth is 0.01 mm into the face: the kernel returns about 5 kN.
3. That force slows the rotor (as a torque of 0.095 x F), pushes the hub back and shoves the wedge.
4. Over the next steps the overlap grows, peaks and shrinks as the bodies respond, and the force follows it.

Element memory

Some elements remember things between steps:

- A **plastic spring** keeps its permanent set. Its kernel (`tabular_return`) checks each step whether the load exceeds the current yield force, and moves the permanent set if it does.
- A **motor drive** keeps the battery's state of charge and the recovery-voltage state of its equivalent circuit.

Only the element's own kernel writes to its memory.

---

6. Hand-written loops (3D and special cases)

Some physics does not fit "one number per body": two robots tumbling in 3D, contact between a deforming blade and a box, or a whole fight with a timed script of spin-ups and pushes. Those sims are written as a loop from a template, using the **same kernels**, in the same order:

```
state  ──►  geometry: where are the contact points?
       ──►  kernels: contact, friction, bending, ESC current, battery voltage
       ──►  apply each force at its point, and the reaction on the partner
       ──►  integrate
```

In 3D the link table becomes an `_apply` helper that puts a force at a point on a body (force plus torque) and the reaction on the other body at the same point.

Data passes between kernels as plain variables. In the battery fight, each step:

```
blade speed        ──►  esc_supply          ──►  weapon power
wheel speed        ──►  motor_current       ──►  drive power
weapon + drive + idle electronics           ──►  total power
total power        ──►  ecm_constant_power  ──►  bus voltage, battery current
bus voltage        ──►  next step's ESCs
battery current    ──►  drains the charge
```

Fast 3D sims also use a few speed tricks that do not change the answer: bigger steps while nothing can touch (proved by a separating-axis gap check), stopping once the outputs are decided, and no memory allocation inside the loop. The spinner sim went from 2.4 s to 0.26 s per engagement with identical results.

---

7. The energy ledger

Every sim keeps books, every step:

```
energy in (motors, batteries, starting spin)
   =  energy stored (springs, kinetic energy, battery)
   +  energy dissipated (damping, friction, plastic work, copper loss, windage)
   +  energy that left (into a target, into the floor)
```

Each element computes its part from its own law, independently of the integration. The mismatch is reported as `_ledger_rel`.

What it catches: a force applied to one body but not its partner, a kernel that creates energy, a contact that returns more energy than it absorbed, or a time step too coarse for a stiff element. A ledger error above 5% fails the sim.

---

8. The harness

The harness wraps the whole sim and runs it many times with changes. It does not look inside. It only asks whether the outputs behave the way a correct model must.

CHECK	WHAT IT DOES	WHAT IT CATCHES
Determinism	Same inputs twice	Hidden randomness, uninitialized memory
Finite and bounded	No NaN or infinity, every output inside physical bounds	Blow-ups, impossible values
Stability margin	Compares the step to the stiffest element's critical step	A step that is about to go unstable
dt convergence	Runs at dt, dt/2, dt/4; estimates the true answer (Richardson)	Answers that depend on the step size. Over 1% fails
Energy ledger	At every step size	Wiring errors, energy sources that should not exist
Limit cases	Inputs with a known answer ("weapon never spins: only idle draw")	Physics that misbehaves at the edges
Knockouts	Turns each physics term off, one at a time	Dead wiring: a term that changes nothing was never connected
Sensitivity	Nudges each uncertain input	Which inputs matter (reported, never judged)
Monte Carlo	Samples the uncertain inputs (small, 20 runs or fewer here)	The spread of the answer
Failure modes	Scores every run against your limits	Probability of failure and safety factors

Two rules the harness never breaks:

1. **No monotonicity checks.** Real behavior is often non-monotonic (wedge damage against plate thickness is one real example), so the harness never expects "more X gives more Y". Trends are reported, not judged.
2. **A failed check means the model gets fixed.** Tolerances are never loosened to make a sim pass. A sim that fails is reported as NOT VERIFIED with the failing checks named.

---

9. Failure modes

Failure is **what you say it is** for that part, never an automatic "over yield". Each mode is written down with its origin:

```
failure:
  - {name: tip drop, metric: tip_drop_mm, limit: 5, fails_when: above, origin: "user: 5 mm
permanent drop"}
```

Every run gets a safety factor per mode (limit / demand). The **governing mode** is the one most likely to fail, with ties broken by the smallest safety factor. A mode the geometry can never reach (a short face that cannot rotate far enough to drop 5 mm) is reported as UNREACHABLE, not as a pass.

---

10. Big studies on your machine

Large Monte Carlo studies are yours to define: which inputs vary, over what ranges, and how many samples. They are written as a **job file**:

```
simblocks_batch: 1
name: spinner_vs_spinner
sim: spinner_vs_spinner.py
entry: batch_record
dt: 2.5e-7
configs: {elastic-plastic (both): {}, rigid blades: {flexH: 0.0, flexV: 0.0}}
samples:
  n: 64
  sampler: sobol
  uniform: {v_close: [1.0, 5.0], y_off: [-0.040, 0.040]}
dt_check: {n: 16, factors: [2, 4]}
verify: {entry: make_spec, mc_n: 20}
```

`simblocks_runner.exe` (or `simblocks batch job.yaml`) runs it on every core, saves progress as it goes so an interruption loses nothing, and packs one `<name>_results.zip` of plain JSON. Sobol sampling spreads points evenly, so a wide space is covered well with tens of samples instead of hundreds.

---

11. Output and the readability gate

Every figure uses the dark mission-control theme. Reports contain:

- **Answer page:** the headline number, one tile per failure mode, safety-factor spreads.
- **Time histories** with your limits drawn in.
- **Design-around page:** which inputs drive the governing margin, and a sweep with the failure band.
- **Verification page:** every harness check with its status.
- **Schematic** of the model as built: bodies, springs, contacts, each labelled with its computed value and the block it came from. Event results (a double hit, a launch) also get a sequence-of-events diagram.

Before anything is saved, each figure goes through an automatic **layout check**: overlapping text, lines through labels, text off the page, fonts under 6 pt. A figure that fails is not saved. Then every image is opened and looked at, because a checker cannot judge crowding or balance.

---

12. Exporting loads to another solver

For a rigid-part FE model (Ansys, MATLAB), the sim exports the force the flexible part actually passes to the attachment point:

- only while contact is active, plus the unloading after it (ringing afterwards is vibration, not load);
- ranked by **signed impulse per direction**, never by total magnitude;
- only cases that pass the time-step convergence check;
- simplified to a short table that keeps the impulse exactly (`fe_load_table`), in your axes.

---

13. Glossary

TERM	MEANING
Body	Something with mass that moves: a rotor, a chassis, a wedge
Element	Something that connects bodies and makes force: a spring, a contact, a motor
Measure	The stretch of an element, written as a sum of body coordinates with lever arms
`C` **table**	All measures stacked as rows; used to compute stretches and to apply forces
Builder	Block that runs once and turns geometry and material into numbers
Kernel	Block that runs every step and turns the current state into a force
Semi-implicit Euler	Update velocity from force, then position from the new velocity; stable for contact
Ledger	The energy books; closure within 5% is required
Knockout	A rerun with one physics term turned off, to prove it was wired in
Richardson	Estimating the zero-step-size answer from runs at dt, dt/2 and dt/4
Sobol	A sampling pattern that covers a space evenly with few points